



Catalyst Optimizer

Dano Lee

Spark Catalyst Optimizer

Spark 1.x, Rule

Spark 2.x, Rule + Cost

Spark 3.0, Rule + Cost + Runtime

Catalyst Optimizer

Catalyst Optimizer

- Spark SQL is Spark's engine for structured data — not just the SQL language. SQL and DataFrame code both run on it.
- Catalyst is its optimizer. It takes your query and rewrites it into a faster one.
- The rewriting happens in steps:
Unresolved Logical Plan → Analyzed → Optimized Logical Plan → Physical Plan
- Spark 1.x — Catalyst rewrites using rules: fixed patterns, applied to every query.
- Spark 2.x — rules can't tell **how many rows survive a filter**. CBO lets Catalyst read table statistics and estimate.

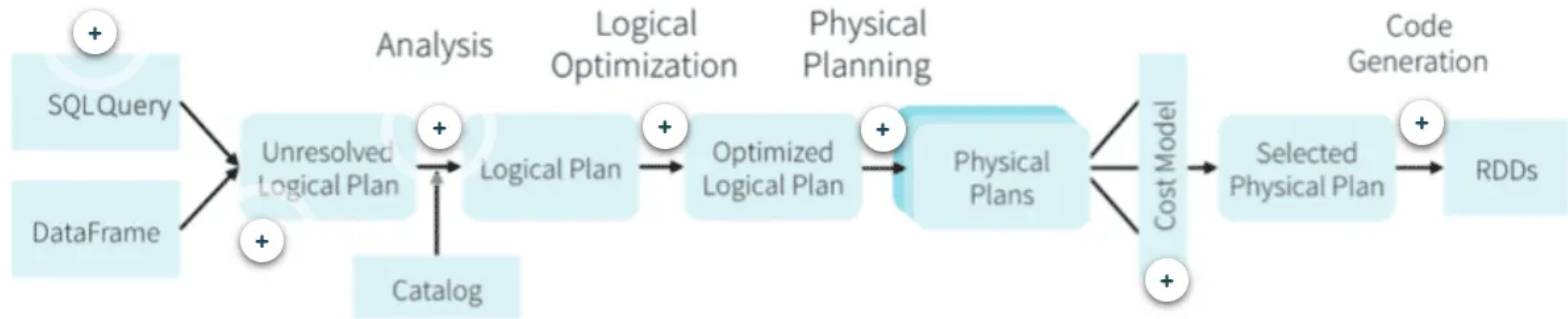
Benefits of Catalyst Optimizer

- **No manual tuning:** write the query in the order that reads best. Catalyst reorders it.
- **Any language, same speed:** SQL, Python, Scala, R all end up as the same plan.
- **Free upgrades:** new Spark versions ship new rules. Old code gets faster.
- **Real code, not interpretation:** Tungsten compiles the final plan into Java code.

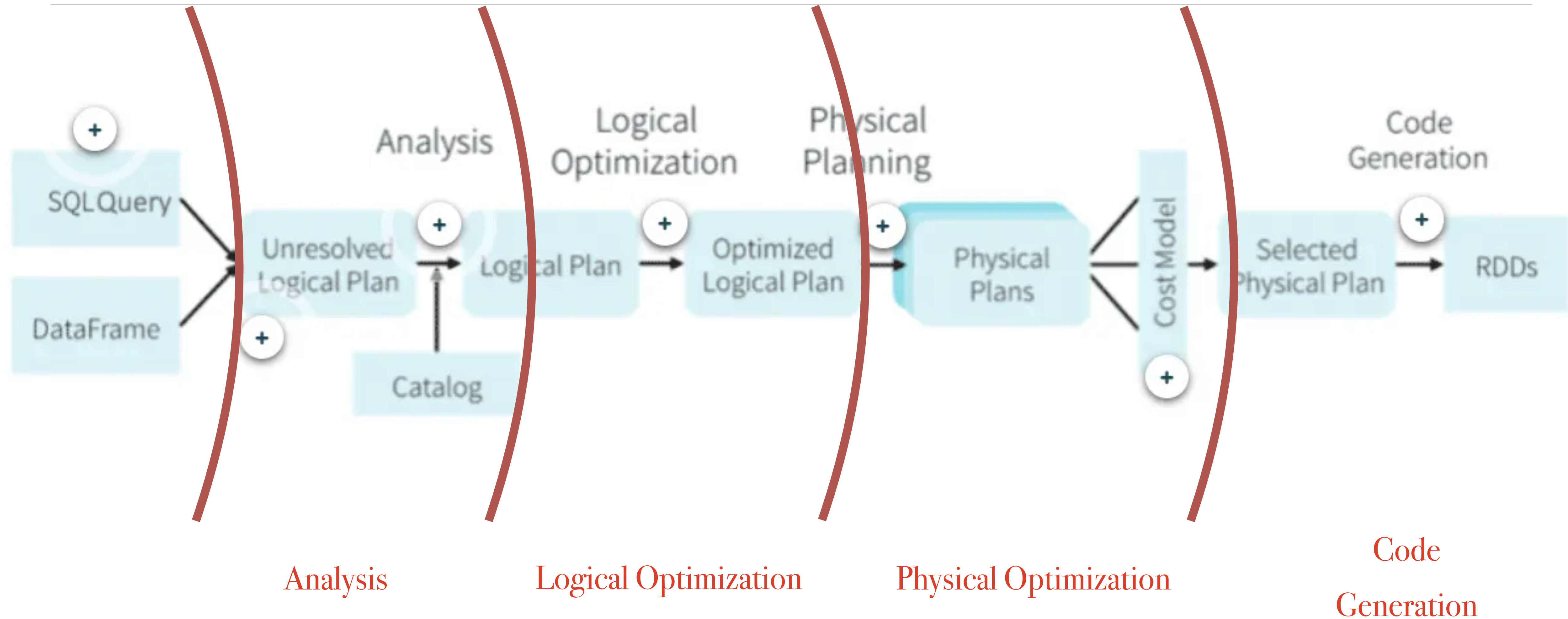
Declarative API

- DataFrame and SQL are declarative APIs. You tell Spark what you intend to do — and NOT how to do it.
- Those two words are the two plans from earlier:
 - **what** → **Logical Plan** — **your intent**
 - **how** → **Physical Plan** — **Spark's decision**
- `df.filter(...).groupBy(...).count()` is a **request**, not a list of instructions. You never wrote *how*. Catalyst fills it in.
- Two ways to give up that freedom:
 - RDD code — you wrote the how yourself. Spark runs it as written.
 - Python UDF — Spark can't see inside your function, so it can't decide the how.

Catalyst Optimizer



Catalyst Optimizer



How Catalyst Works Internally (1): Logical Plans

- **Unresolved Logical Plan:** your query becomes a tree of operators (Filter, Project, Join).
 - SQL text is parsed into the tree. DataFrame code builds the tree directly.
 - "Unresolved" = Spark doesn't know yet whether *passenger_count* is a real column.
- **Analyzed Logical Plan:** Catalyst checks the tree against the Catalog.
 - Does the table exist? Does the column exist? What type is it?
 - This is where *AnalysisException* comes from. The query never runs.
- **Optimized Logical Plan:** Catalyst applies rules until nothing changes anymore.
 - **Constant Folding** — *age > 20+10* becomes *age > 30*. Computed once, not per row.
 - **Predicate Pushdown** — move filters down toward the data. Read less.
 - **Column Pruning** — drop columns nobody asked for. Read less.

How Catalyst Works Internally (2): Physical Plan

- ❖ **Physical Plan:** every logical operator is replaced by a real Spark operator.
 - ❖ Join is an intention. BroadcastHashJoin is code that actually runs.
 - ❖ Which one? Mostly a size decision: if one side fits under `spark.sql.autoBroadcastJoinThreshold` (10MB), broadcast it. If not, sort-merge.
- ❖ **Preparation:** Catalyst inserts whatever the chosen operators need.
 - ❖ Sort-merge join needs both sides sorted and co-located → Catalyst adds Exchange (shuffle) and Sort.
 - ❖ This is where the shuffles from Week 5 come from. You never wrote them.
- ❖ **Code Generation:** the final plan becomes Java code, then RDDs
 - ❖ Handled by Tungsten, Spark's execution layer.
 - ❖ The plan you read in `explain()` is not what runs. The generated code is.

Reading the Plans with `.explain(True)`

```
df = spark.read.parquet("trips")  
df.filter(col("passenger_count") > 1).select("trip_distance").explain(True)
```


== Parsed Logical Plan == ← **Unresolved**

'Project ['trip_distance]

+ - 'Filter ('passenger_count > 1)

 + - Relation [VendorID,passenger_count,trip_distance,...] parquet

== Analyzed Logical Plan == ← **Catalog checked, types resolved**

trip_distance: double

Project [trip_distance#12]

+ - Filter (passenger_count#8 > 1)

 + - Relation [VendorID#5,passenger_count#8,trip_distance#12,...] parquet

== Optimized Logical Plan == ← **Rules applied**

Project [trip_distance#12]

+ - Filter (isnotnull(passenger_count#8) AND (passenger_count#8 > 1))

 + - Relation [passenger_count#8,trip_distance#12] parquet

== Physical Plan == ← **Real operators**

*(1) Project [trip_distance#12]

+ - *(1) Filter (isnotnull(passenger_count#8) AND (passenger_count#8 > 1))

 + - FileScan parquet [passenger_count#8,trip_distance#12]

 PushedFilters: [IsNotNull(passenger_count), GreaterThan(passenger_count,1)]

Reading the Plans with `.explain(True)`

- **Four blocks:** the four steps we just walked through.
- **Analyzed:** 'passenger_count' → passenger_count#8. Spark checked the Catalog and confirmed the column exists. The #8 is its internal ID.
- **Optimized:** the Relation [...] list drops from every column to just the 2 we need. Parquet skips the rest on disk — column pruning, visible.
- **Physical:** the filter also appears as PushedFilters inside FileScan. Parquet skips whole chunks that can't match, so fewer rows ever reach Spark — predicate pushdown, visible.

Reading the Plans with `.explain(True)`

- **Four blocks:** the four steps we just walked through.
- **Analyzed:** 'passenger_count' → passenger_count#8. Spark checked the Catalog and confirmed the column exists. The #8 is its internal ID.
- **Optimized:** the Relation [...] list drops from every column to just the 2 we need. Parquet skips the rest on disk — column pruning, visible.
- **Physical:** the filter also appears as PushedFilters inside FileScan. Parquet skips whole chunks that can't match, so fewer rows ever reach Spark — predicate pushdown, visible.

== Parsed Logical Plan == ← Unresolved

```
'Project [trip_distance]
+- 'Filter ('passenger_count > 1)
   +- Relation [VendorID,passenger_count,trip_distance,...] parquet
```

== Analyzed Logical Plan == ← Catalog checked, types resolved

```
trip_distance: double
Project [trip_distance#12]
+- Filter (passenger_count#8 > 1)
   +- Relation [VendorID#5,passenger_count#8,trip_distance#12,...] parquet
```

Spark 내부의 컬럼 고유 ID(exprId)

== Optimized Logical Plan == ← Rules applied

```
Project [trip_distance#12]
+- Filter (isnotnull(passenger_count#8) AND (passenger_count#8 > 1))
   +- Relation [passenger_count#8,trip_distance#12] parquet
```

== Physical Plan == ← Real operators

```
*(1) Project [trip_distance#12]
+- *(1) Filter (isnotnull(passenger_count#8) AND (passenger_count#8 > 1))
   +- FileScan parquet [passenger_count#8,trip_distance#12]
      PushedFilters: [IsNotNull(passenger_count), GreaterThan(passenger_count,1)]
```

Reading the Plans with `.explain(True)`

- **Four blocks:** the four steps we just walked through.
- **Analyzed:** 'passenger_count' → passenger_count#8. Spark checked the Catalog and confirmed the column exists. The #8 is its internal ID.
- **Optimized:** the Relation [...] list drops from every column to just the 2 we need. Parquet skips the rest on disk — column pruning, visible.
- **Physical:** the filter also appears as PushedFilters inside FileScan. Parquet skips whole chunks that can't match, so fewer rows ever reach Spark — predicate pushdown, visible.

== Parsed Logical Plan == ← Unresolved

'Project ['trip_distance]

+ - 'Filter ('passenger_count > 1)

+ - Relation [VendorID,passenger_count,trip_distance,...] parquet

== Analyzed Logical Plan == ← Catalog checked, types resolved

trip_distance: double

Project [trip_distance#12]

+ - Filter (passenger_count#8 > 1)

+ - Relation [VendorID#5,passenger_count#8,trip_distance#12,...] parquet

== Optimized Logical Plan == ← Rules applied

Project [trip_distance#12]

+ - Filter (isnotnull(passenger_count#8) AND (passenger_count#8 > 1))

+ - Relation [passenger_count#8,trip_distance#12] parquet

Column Pruning

== Physical Plan == ← Real operators

*(1) Project [trip_distance#12]

+ - *(1) Filter (isnotnull(passenger_count#8) AND (passenger_count#8 > 1))

+ - FileScan parquet [passenger_count#8,trip_distance#12]

PushedFilters: [IsNotNull(passenger_count), GreaterThan(passenger_count,1)]

== Parsed Logical Plan == ← Unresolved

'Project ['trip_distance]
+- 'Filter ('passenger_count > 1)
 +- Relation [VendorID,passenger_count,trip_distance,...] parquet

== Analyzed Logical Plan == ← Catalog checked, types resolved

trip_distance: double
Project [trip_distance#12]
+- Filter (passenger_count#8 > 1)
 +- Relation [VendorID#5,passenger_count#8,trip_distance#12,...] parquet

== Optimized Logical Plan == ← Rules applied

Project [trip_distance#12]
+- Filter (isnotnull(passenger_count#8) AND (passenger_count#8 > 1))
 +- Relation [passenger_count#8,trip_distance#12] parquet

== Physical Plan == ← Real operators

*(1) Project [trip_distance#12]
+- *(1) Filter (isnotnull(passenger_count#8) AND (passenger_count#8 > 1))
 +- FileScan parquet [passenger_count#8,trip_distance#12]
 PushedFilters: [IsNotNull(passenger_count), GreaterThan(passenger_count,1)]

Spark checked the Catalog and confirmed the column exists

Reading the Plans with `.explain(True)`

- **Four blocks:** the four steps we just walked through.
- **Analyzed:** `'passenger_count' → passenger_count#8`. Spark checked the Catalog and confirmed the column exists. The `#8` is its internal ID.
- **Optimized:** the Relation [...] list drops from every column to just the 2 we need. Parquet skips the rest on disk — column pruning, visible.
- **Physical:** the filter also appears as `PushedFilters` inside `FileScan`. Parquet skips whole chunks that can't match, so fewer rows ever reach Spark — predicate pushdown, visible.

== Parsed Logical Plan == ← Unresolved

'Project ['trip_distance]

+ - 'Filter ('passenger_count > 1)

+ - Relation [VendorID,passenger_count,trip_distance,...] parquet

== Analyzed Logical Plan == ← Catalog checked, types resolved

trip_distance: double

Project [trip_distance#12]

+ - Filter (passenger_count#8 > 1)

+ - Relation [VendorID#5,passenger_count#8,trip_distance#12,...] parquet

== Optimized Logical Plan == ← Rules applied

Project [trip_distance#12]

+ - Filter (isnotnull(passenger_count#8) AND (passenger_count#8 > 1))

+ - Relation [passenger_count#8,trip_distance#12] parquet

== Physical Plan == ← Real operators

*(1) Project [trip_distance#12]

+ - *(1) Filter (isnotnull(passenger_count#8) AND (passenger_count#8 > 1))

+ - FileScan parquet [passenger_count#8,trip_distance#12]

PushedFilters: [IsNotNull(passenger_count) GreaterThan(passenger_count,1)]

Filter was above Relation. - also pushed into FileScan, so only what's needed gets read.

Why the Filter Appears Twice

```
== Physical Plan ==          ← Real operators
*(1) Project [trip_distance#12]
+ *(1) Filter (isnotnull(passenger_count#8) AND (passenger_count#8 > 1))
+- FileScan parquet [passenger_count#8,trip_distance#12]
   PushedFilters: [IsNotNull(passenger_count), GreaterThan(passenger_count,1)]
```

- Parquet stores rows in row groups — chunks sized by bytes — 128MB by default, often hundreds of thousands of rows.
- **PushedFilters** reads only those min/max values and decides whether to open the chunk at all.

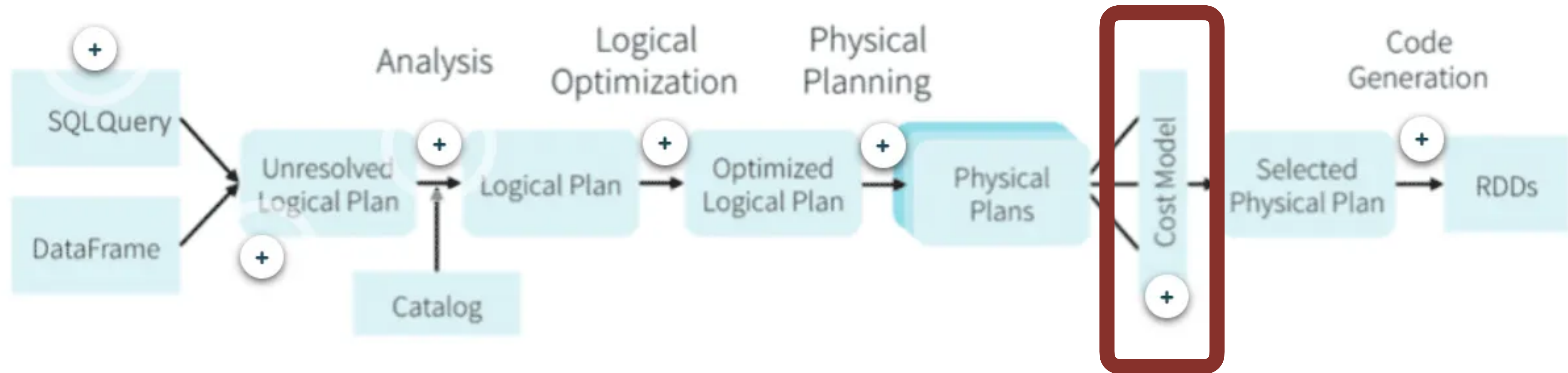
Chunk	passenger_count min–max	Result
A	1 – 1	skipped, never opened
B	1 – 4	opened, all rows sent to Spark

Why the Filter Appears Twice

```
== Physical Plan ==          ← Real operators
*(1) Project [trip_distance#12]
+ *(1) Filter (isnotnull(passenger_count#8) AND (passenger_count#8 > 1))
+- FileScan parquet [passenger_count#8,trip_distance#12]
   PushedFilters: [IsNotNull(passenger_count), GreaterThan(passenger_count,1)]
```

- Chunk B still contains rows where `passenger_count = 1`. The Filter above removes them.
- Not duplicated work — skipping chunks is cheap and rough, filtering rows is exact.

Cost Model in Catalyst Optimizer



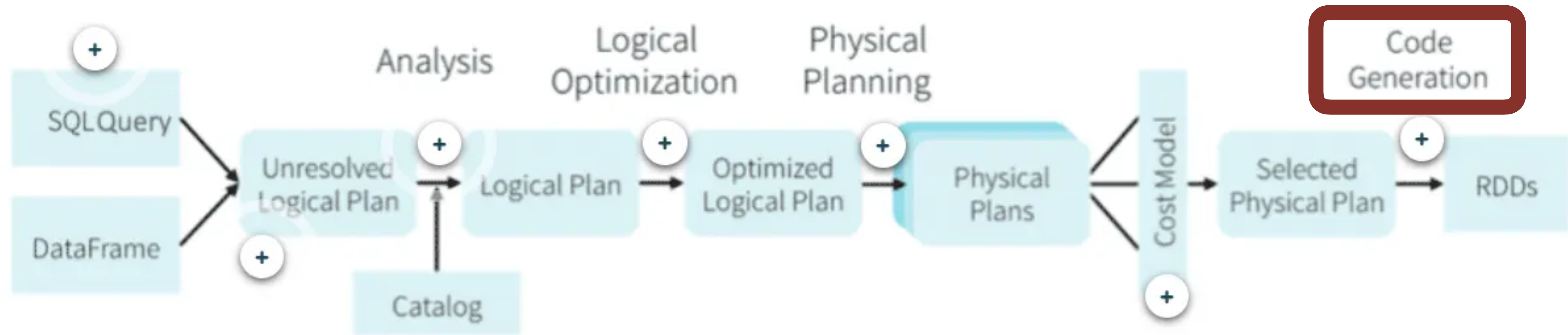
Cost-Based Optimization (CBO)

- Rules are blind to your data. *PushDownPredicates* fires the same way whether the table has 100 rows or 100 billion.
- But some decisions need numbers. Joining three tables — which two go first? Depends on how big each one is.
- Spark 2.2 added CBO: Catalyst reads statistics from the Catalog and uses them to decide.
- Statistics = row count, table size, and per column: distinct values, min, max, nulls.
- Someone has to collect them first. Off by default — *spark.sql.cbo.enabled = false*.

What the Numbers Actually Decide

- **Join order** — with three tables, join the pair that produces the smallest intermediate result first. Fewer rows carried into the next join. (*spark.sql.cbo.joinReorder.enabled*)
- **Join strategy** — broadcast or sort-merge? This one is a size check, not a cost calculation:
 - one side under *spark.sql.autoBroadcastJoinThreshold* (10MB) → *BroadcastHashJoin*
 - otherwise → *SortMergeJoin*
- The diagram shows a Cost Model box choosing between physical plans. In practice Spark doesn't rank whole plans — it picks operators by rules and size thresholds. Statistics make those size estimates better.

Code Generation in Catalyst Optimizer



Whole-Stage Code Generation

- a key feature of Apache Spark's execution engine, part of the **Tungsten** project.
- responsible for generating optimized Java bytecode that represents **the entire set of transformations within a stage as a single, efficient function.**
(not the scheduling stage learned in W6)
- This approach **removes the function call that happens for every row at every operator**, and keeps data in CPU registers instead of moving it back and forth — ultimately improving performance.

Whole-Stage Code Generation

```
result = (  
    df.filter(col("passenger_count") > 1) # Filter rows  
    .select("trip_distance")             # Keep one column  
)
```

- Spark combines these transformations into a single Java function. The code is optimized to minimize CPU instructions, data movement, and function call overhead, and the entire pipeline is executed as a tight loop over the data.

Before Codegen: The Volcano Model

```
Project.next()
└─ Filter.next()
    └─ Scan.next() → one row
    check passenger_count > 1
    failed? ask Scan again
    pick trip_distance
```

- Every operator is a separate object with a next() method. The one above asks the one below for a row.
- One row = one full chain of calls. 100 million rows means 300 million calls.
- Each operator is generic. The Filter object doesn't know your condition is `passenger_count > 1` — it was written to handle any condition on any column.
- Rows must be materialized in memory to pass between operators. Nothing stays in CPU registers.

After Codegen: One Loop

```
// Conceptually, the chain collapses into this
while (rows.hasNext()) {
    row = rows.next();
    if (row.passenger_count <= 1) continue; // Filter, inlined
    output(row.trip_distance);           // Project, inlined
}
```

- Catalyst writes a Java function for the whole chain — with your condition compiled straight into it.

	Volcano	Codegen
Calls per row	one per operator	one, at the scan
Operator code	generic	written for this query
Row data	in memory	in CPU registers

Seeing the Generated Code

```
result.explain("codegen")
```

Dynamic Partition Pruning

- ❖ **Dynamic Partition Pruning = Predicate Push Down + Broadcast Hash Join**
- ❖ **`spark.sql.optimizer.dynamicPartitionPruning.enabled`**
(on by default since Spark 3.0.)

Example of DPP

- ❖ Imagine you have a **fact table with sales records (Sales)** and a **dimension table with dates (Date)**.
- ❖ If you **filter the Date table to only include January 2024 dates**, Spark will use DPP to **only scan partitions in the Sales table that contain records from January 2024**, instead of scanning the entire Sales table.

Example of DPP

Sales Table (Partitioned by `sale_date`)

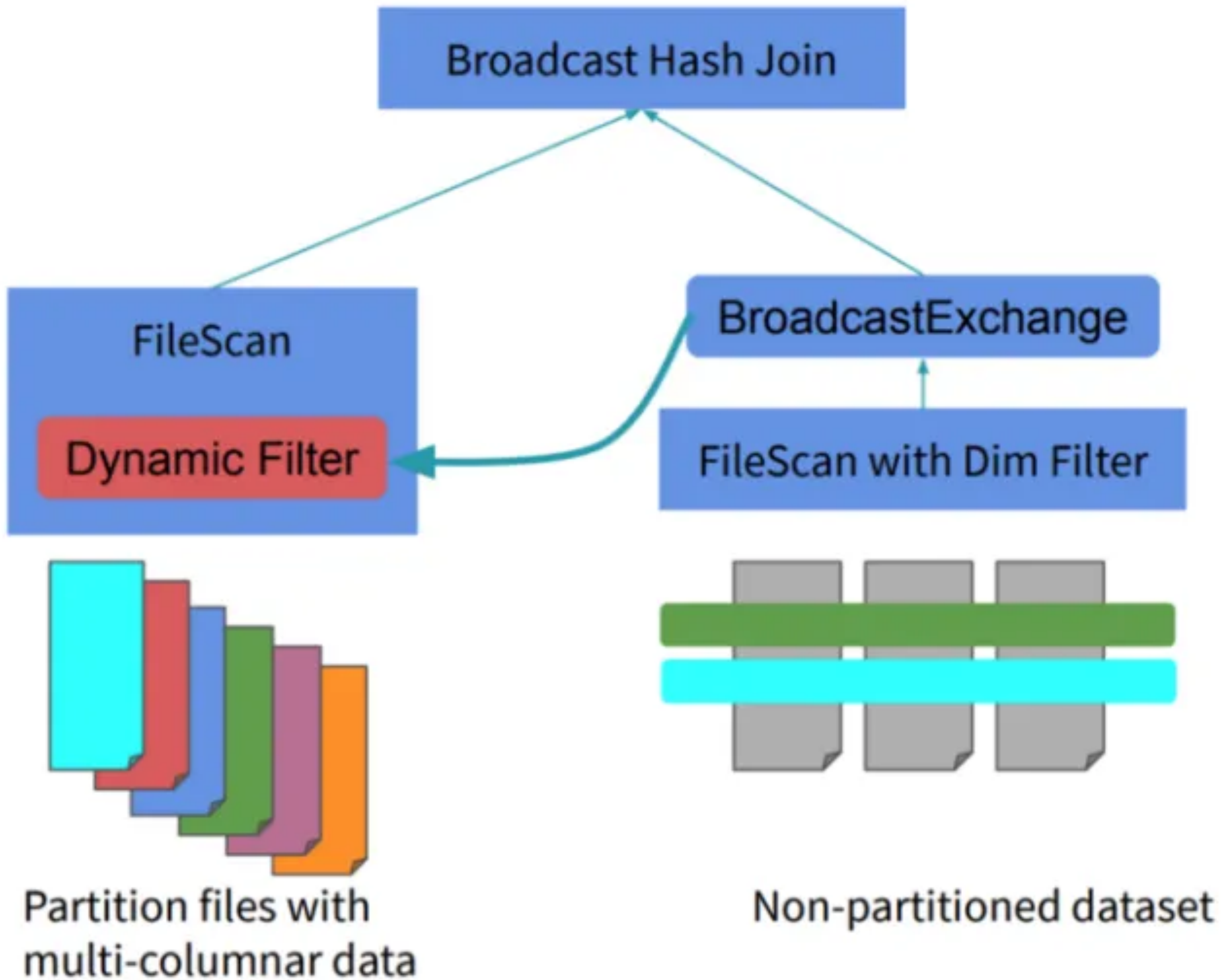
sale_id	product	amount	sale_date
1	A	100	2024-01-01
2	B	150	2024-01-05
3	C	200	2024-02-10
4	D	250	2024-02-15
5	E	300	2024-03-01

Date Table (Dimension)

date	month	year
2024-01-01	Jan	2024
2024-01-02	Jan	2024
2024-01-03	Jan	2024
2024-01-04	Jan	2024
...	...	...
2024-02-01	Feb	2024
2024-02-02	Feb	2024
...	...	...

Dynamic Partition Pruning

- ❖ The smaller(**dimension**) table is queried and filtered. A **hash table** is built as part of the filter query.
- ❖ Spark uses the result of this query (and hash table) to create a **broadcast variable**.
- ❖ Then, it will broadcast the filter to each executor.
- ❖ Spark inserts a **dynamic filter** into the scan of the bigger (**fact**) table — an internal subquery built from the filter on the smaller table.
- ❖ When Spark scans the larger table, it applies this filter and **skips partitions that don't match**, reducing the data read.



Constraints of Dynamic Partition Pruning

- ❖ The fact table **must be partitioned on one of the join key columns**.
- ❖ Works only with **equi-joins** (= conditions).
- ❖ By default the dimension side must be **broadcastable** — under *autoBroadcastJoinThreshold*. This is the most common reason DPP silently doesn't fire.
- ❖ Most effective on **star schemas**: a large partitioned fact table with selective filters on small dimension tables.